

BEST PRACTICES: TAMSANG

ชื่อไฟล์: LOWERCASE

- **lowerCamelCase:** ใช้สำหรับ **Variables, Functions,** และ **Methods**
 - ตัวอย่าง: `const userEmail = 'test@example.com';`
- **UpperCamelCase (PascalCase):** ใช้สำหรับ **Classes** หรือ **Constructor Functions**
 - ตัวอย่าง: `class UserAdmin { ... }`
- **UPPER_SNAKE_CASE:** ใช้สำหรับ **Constants** (ค่าคงที่ที่ไม่มีการเปลี่ยนแปลงตลอดโปรแกรม)
 - ตัวอย่าง: `const MAX_RETRY_LIMIT = 5;`

เลิกใช้ชื่อตัวแปรตัวเดียว (Avoid Single Letters)

ยกเว้นใน `loop` สั้นๆ (เช่น `i`, `j`) นอกนั้นควรสื่อความหมาย

- ❌ `const d = new Date();`
- ✅ `const currentDate = new Date();`

ใช้คำกริยานำหน้าสำหรับ Boolean (Prefix Booleans)

เพื่อให้รู้กันทีว่าเป็นคำถาม Yes/No

- นิยมใช้: `is`, `has`, `can`, `should`
- ตัวอย่าง: `isLoggedIn`, `hasPermission`, `isValidEmail`

ใช้คำกริยาสำหรับฟังก์ชัน (Functions should be Actions)

ฟังก์ชันคือการกระทำ จึงควรขึ้นต้นด้วยคำกริยา

- ตัวอย่าง: `getUserData()`, `calculateTotal()`, `validateInput()`

```
describe('remove user', () => {  
  test('Remove one', () => {  
    testUserData = new UserData();  
    const userCountBefore = testUserData.getUserCount();  
    testUserData.removeOneUser();  
    const userCountAfter = testUserData.getUserCount();  
    const difUser = userCountBefore - userCountAfter;  
    expect(difUser).toEqual(1);  
  })  
})
```

```
/**
 Represents a Expense.
 @constructor
 @param {date} date - The date of the expense.
 @param {float} income - The author of the book.
 @param {float} expense - This is expense.
 @param {string} detail - This is detail.
 */
constructor(date, income, expense, detail) {
  this.date = date;
  this.income = parseFloat(income) || 0;
  this.expense = parseFloat(expense) || 0;
  this.detail = detail || '';
}
```

Commit message ขึ้นต้นด้วย ticket id เช่น ถ้า ticket id is 32,
commit message ใช้ “32: add for loop to count contents”
Branch name ให้ขึ้นต้นด้วย your name id + _ +ticket id เช่น
werayuthswu_32